

Section 88 — Formal Power Series

Polynomials as locally determined infinite objects

This lesson derives Problems A–C. Problem D is deliberately left for the editorial.

A formal power series is a coefficient sequence. Equality modulo x^n means that the first n coefficients agree; convergence and numerical approximation play no role. The modulus $998244353 = 119 * 2^{23} + 1$ supplies the roots of unity needed by NTT, so a product of length n costs $M(n) = O(n \log n)$.

A. Inverse: lift twice as many correct coefficients

Suppose $A G = 1 \bmod x^m$. Write the error as $E = 1 - A G$; its first m coefficients vanish. Newton's update is

$$G' = G * (2 - A * G).$$

Then $1 - A G' = (1 - A G)^2 = E^2$, whose first $2m$ coefficients vanish. Truncate every temporary polynomial to the precision currently requested. Starting from $G[0] = \text{inverse}(A[0])$, doubling gives the answer in $O(M(n))$, because the geometric sum $M(1) + M(2) + \dots + M(n)$ is $O(M(n))$.

B. Logarithm: turn multiplication into addition

The formal derivative obeys the familiar product rule, so

$$\begin{aligned} \log(A)' &= A' / A. \\ \log(A) &= \text{integral}(A' * \text{inverse}(A)). \end{aligned}$$

The constant is fixed to zero, which is why the input requires $A[0]=1$. Integration divides coefficient $i-1$ by i ; all needed integers are invertible because $n < \text{MOD}$. This pattern is useful whenever a combinatorial construction turns products into sums of connected components.

C. Exponential: Newton on $\log(G)=A$

Assume $\log(G)=A \bmod x^m$. Correct it with

$$G' = G * (1 - \log(G) + A).$$

The parenthesis has constant coefficient one and cancels the known logarithmic error. A formal expansion shows the new error starts at degree $2m$. Begin with $G=1$, forced by $A[0]=0$, and double precision.

How to recognize the method

First write the algebraic identity defining the unknown series. Ask whether its constant term determines a base coefficient. Next obtain a correction whose error is squared; that is the Newton signal. Finally audit every truncation: coefficients at degree at least n can never influence a result below degree n .

Precision is part of the invariant

If G is correct modulo x^m , that means coefficients 0 through $m-1$ are correct. A multiplication used to lift precision to $2m$ should therefore be truncated to $\min(2m, n)$. Keeping more coefficients wastes time; keeping fewer can silently destroy the proof.

For example, let $A=1+x$ and begin with $G=1$. Then

$$\begin{aligned} G * (2-A*G) &= 1-x && \text{modulo } x^2 \\ (1-x) * (2-(1+x)(1-x)) &&& \\ &= 1-x+x^2-x^3 && \text{modulo } x^4. \end{aligned}$$

The visible coefficients are the start of $1/(1+x)$. Each update doubles the known prefix; it does not double the polynomial's degree for its own sake.

Preconditions are mathematical, not cosmetic

- `inverse(A)` exists exactly when $A[0]$ is nonzero.
- The normalized logarithm requires $A[0]=1$.
- The normalized exponential requires $A[0]=0$.
- A square root needs a compatible constant term and a chosen branch; the usual Newton division also needs the chosen root to be nonzero.
- Formal integration through degree $n-1$ needs $1, \dots, n-1$ invertible in the coefficient field.

Engineering and tests

Make multiplication responsible for choosing an NTT length at least the full convolution length, then resize the returned polynomial explicitly. Normalize negative coefficients after every subtraction. Reuse inverse tables and NTT buffers only if their lifetime and required precision are clear.

Test every operation by substituting it back into its defining identity: $A*\text{inv}(A)=1$, derivative of $\log(A)$ equals A'/A , $\log(\exp(A))=A$, and a reported square root squares to A , all at the requested precision. Random small naive convolution is a stronger oracle than a few hand-written outputs.

Exercises

A–C build inverse, logarithm, and exponential. D asks you to transfer the same correction idea to a square root.